

An effective multi-start iterated greedy algorithm to minimize makespan for the distributed permutation flowshop scheduling problem with preventive maintenance

Jia-yang Mao^a, Quan-ke Pan^{a,*}, Zhong-hua Miao^a, Liang Gao^b

^a School of Mechatronic Engineering and Automation, Shanghai University, Shanghai 200072, PR China

^b The State Key Laboratory of Digital Manufacturing Equipment and Technology, Huazhong University of Science & Technology, Wuhan, PR China

ARTICLE INFO

Keywords:

Distributed permutation flowshop scheduling problem
Makespan
Preventive maintenance
Iterated greedy algorithm
Heuristic methods

ABSTRACT

In recent years, distributed scheduling problems have been well studied for their close connection with multi-factory production networks. However, the maintenance operations that are commonly carried out on a system to restore it to a specific state are seldom taken into consideration. In this paper, we study a distributed permutation flowshop scheduling problem with preventive maintenance operation (PM/DPFSP). A multi-start iterated greedy (MSIG) algorithm is proposed to minimize the makespan. An improved heuristic is presented for the initialization and re-initialization by adding a dropout operation to NEH2 to generate solutions with a high level of quality and dispersiveness. A destruction phase with the tournament selection and a construction phase with an enhanced strategy are introduced to avoid local optima. A local search based on three effective operators is integrated into the MSIG to reinforce local neighborhood solution exploitation. In addition, a restart strategy is adopted if a solution has not been improved in a certain number of consecutive iterations. We conducted extensive experiments to test the performance of the presented MSIG. The computational results indicate that the presented MSIG has many promising advantages in solving the PM/DPFSP under consideration.

1. Introduction

The distributed permutation flowshop scheduling problem (DPFSP) has attracted increasing attention (Fernandez-Viagas & Framinan, 2015; Pan et al., 2019) in the context of decentralized and globalized economy. A multi-factory production environment is becoming more and more common. In multi-factory production network, the total manufacture tasks are allocated among several independent units, which brings great benefits to the enterprise, including higher production efficiency, lower production costs, better product quality, and so on (Behnamian & Ghomi, 2016). The DPFSP explores the production scheduling problem under the condition that companies have established factories in different locations.

DPFSP contains n jobs in the set $N = \{1, 2, \dots, n\}$ that is to be assigned to one factory out of a set of F identical factories $G = \{1, 2, \dots, F\}$, in which each factory contains the same series of m machines $M = \{1, 2, \dots, m\}$. It is assumed that all the machines are continuously available from the beginning to the end in the production process. Nevertheless, in the actual industrial production, reliability and maintainability decrease as

the operation time of the machine increases (Rausand & Høyland, 2003). Generally, companies are aiming at a reliable production system with high availability performance. Therefore, maintenance operations are carried out on a system to restore it to a specific state (Briolini, 2013). There are two large types of maintenance operations: corrective maintenance (CM) and preventive maintenance (PM). The actions carried out after the failure has already occurred are referred to as CM. PM operations are carried out on machines before the breakdown or failure takes place. The purpose of PM operations is to increase equipment life and to enhance production management during production operations. The advantage of PM is that machines are always in good condition, thus reducing the risk of failures and avoiding the loss of failure. In industry, PM operations are widely used and planned to be conducted at pre-defined time intervals without considering specific working conditions of equipment.

In this paper, the DPFSP with PM operation (PM/DPFSP) is considered. A complete schedule of the PM/DPFSP involves two subproblems: the assignment of jobs to F factories and the sequence of jobs in each factory. The PM/DPFSP is an NP-Hard problem as a variation of DPFSP

* Corresponding author.

E-mail addresses: maojy1996@qq.com (J.-y. Mao), panquanke@shu.edu.cn (Q.-k. Pan), zhhmiao@shu.edu.cn (Z.-h. Miao), gaoliang@mai.hust.edu.cn (L. Gao).

<https://doi.org/10.1016/j.eswa.2020.114495>

Received 2 September 2020; Received in revised form 16 November 2020; Accepted 10 December 2020

Available online 15 December 2020

0957-4174/© 2020 Elsevier Ltd. All rights reserved.

(Naderi & Ruiz, 2010). The objective is to minimize the maximum complete time C_{max} among all the factories. In order to obtain solutions with both good quality and high efficiency, we propose a multi-start iterated greedy (MSIG) algorithm in this paper. The MSIG is a meta-heuristic algorithm that obtains only one candidate solution in each iteration. To generate an initial solution with both good quality and diversity, we apply an improved NEH2 heuristic method with dropout operations. Then in the destruction and construction phases, a few jobs are randomly removed from the current solution and then greedily reinserted into new positions to explore neighboring solutions. Afterward, we consider two operators to propose three local search methods to constitute an efficient local search procedure. Finally, we utilize a simulated annealing acceptance criterion and a restart strategy to prevent local optimum. The purpose of the restart strategy is to reinitialize the current solution after it has not been improved for several iterations.

The remaining components of this paper are organized as follows. The next section reviews the relevant literature. Section 3 presents the PM/DPFSP. The proposed MSIG is illustrated in Section 4. Numerical test experiments on MSIG are provided in Section 5. Finally, Section 6 summarizes conclusions and provides some topics of future research.

2. Literature review

The closely related literature can be divided into two categories: DPFSP and the scheduling problems with PM operation. In order to narrow the gap between academic research and practical applications, DPFSP has been studied for a long time as an extension of the permutation flowshop scheduling problem (Maccarthy and Liu, 1993). Naderi and Ruiz (2010) first denoted DPFSP as $DP/prmu/C_{max}$, and proposed a total of 12 heuristic methods and two variable neighborhood descent (VND) algorithms. Following the initial work of Naderi and Ruiz (2010), many authors have studied more advanced algorithms in the last decades. A hybrid genetic algorithm was presented by Gao and Chen (2011) to minimize the maximum completion time, where an efficient local search was utilized to explore neighboring solutions. Gao, Chen and Deng (2013) proposed a tabu algorithm with a novel tabu strategy. They utilized an enhanced local search method and a sub-sequence swapping method to generate neighborhood solutions. Lin, Ying and Huang (2013) modified the iterated greedy (IG) algorithm for the DPFSP, and proposed a destruction operator with a variable number of removed jobs and a simulated annealing algorithm with a sinking temperature. Wang et al. (2013) proposed an effective estimation of distribution algorithm (EDA). They employed the earliest completion factory rule, and designed a probability model, and proposed four local search operators based on the problem characteristics. Naderi and Ruiz (2014) presented a scatter search (SS) procedure, which has seldom been explored for flowshop settings. The SS procedure was demonstrated to statistically outperform the other 11 algorithms in the experiments. Xu et al. (2014) proposed a hybrid immune algorithm, where four different local search operators were used in a hybrid way within the frame of the immune algorithm. Fernandez-Viagas and Framinan (2015) developed a bounded-search IG algorithm for the DPFSP. In order to reduce the number of solutions to be evaluated, they studied the property of the problem to estimate the makespan of a factory when a new job is inserted. The effect of the bounded search method is that the computational time is discarded by 32.31% and the number of iterations is decreased by 11.68%. Bargaoui, Belkahla Driss and Ghédira (2017) presented a chemical reaction optimization algorithm, in which a one-point crossover and a greedy strategy are embedded to ameliorate the solution quality. Deng and Wang (2017) proposed a competitive memetic algorithm to solve the multi-objective DPFSP with makespan and total tardiness criteria. They employed two populations to optimize the two objectives and designed the interaction between the two populations to balance the two objectives. Ruiz, Pan and Naderi (2019) presented a simple two-stage iterated greedy (IG2S) algorithm with improved initialization, destruction, construction and local search

procedures. Despite its simplicity, the IG2S significantly exceeds the other state-of-the-art algorithms in comparison. For the DPFSP with total flowtime optimization, Fernandezviagas, Perezgonzalez and Framinan (2018) proposed eighteen constructive heuristics to generate feasible solutions with high quality in reasonable computational time and an evolutionary algorithm to further improve the so-obtained solutions. Pan et al. (2019) introduced three constructive heuristics and four metaheuristics to suit the needs of different CPU time demands and solution quality.

There are also literatures proposed for other variations of the DPFSP. Rifai, Nguyen and Dawal (2016) gave a survey of the distributed reentrant permutation flowshop scheduling problem and developed a multi-objective adaptive large neighborhood search algorithm. Wang and Wang (2016) proposed a memetic algorithm for solving the distributed assembly permutation flowshop. Wang, Huang and Qin (2016) considered the DPFSP under machine breakdown with makespan criterion and proposed a fuzzy logic-based hybrid EDA. Zhang, Xing and Cao (2018) presented a hybrid discrete differential evolution (DDE) algorithm to deal with the distributed blocking flowshop scheduling problem (DBFSP) with makespan criterion. Pan et al. (2019) addressed a distributed assembly permutation flowshop scheduling problem in which each factory consists of a flowshop for part-processing and an assembly line for product-processing. Meng, Pan and Wang (2019) studied a DPFSP with customer order constraint. Jing et al. (2020) dealt with the DPFSP with due windows to minimize the total weighted earliness and tardiness. Huang, Pan and Gao (2020) extend the DPFSP by considering the sequence-dependent setup time (SDST). Zhao et al. (2020) presented an ensemble DDE algorithm for solving the DBFSP with makespan minimization. Meng and Pan (2021) solved a DPFSP with non-identical factories (DPFSP-NF) with lot-streaming and SDST considered. Lu et al. (2021) presented a knowledge-based multi-objective memetic optimization algorithm to minimizing makespan, negative social impact and total energy consumption of the DPFSP-NF. Wang and Wang (2020) applied a knowledge-based cooperative algorithm to address the energy-efficient scheduling of the DPFSP. Most recently, Huang et al. (2021) proposed three heuristic methods and an effective DABC algorithm for minimizing the makespan of the DPFSP with the machine-independent SDST.

The corresponding literatures on the scheduling problem with PM operation are abundant as well. Ruiz, Garciadiaz and Maroto (2007) considered PM operation in a flowshop scheduling problem. They compared three PM policies and tested three heuristics and two meta-heuristic methods. Extensive computational evaluations have given a result that the genetic algorithm (GA) and the ant colony optimization algorithm (ACO) are the two best algorithms. Yang, Hsu and Kuo (2008) considered a two-machine flowshop scheduling problem with a separated maintenance constraint. According to their problem, the machine needs PM operations after it completes a certain amount of jobs rather than keeps working for a certain period of time. Wang and Yu (2010) proposed a filtered beam search based heuristic algorithm for solving a flexible job-shop scheduling problem with machine availability constraints. They studied a scheduling problem with both fixed and non-fixed machine availability constraints in the flexible job-shop manufacturing environment. Naderi, Zandieh and Amimmayeri (2011) investigated a flexible flowshop scheduling with periodic PM operation on machines to minimize makespan. They applied a GA and an artificial immune system to solve the problem. Wong, Chan and Chung (2013) formulated a new scheduling problem with multiple resources and PM tasks as an integer-programming model and solved it by a GA approach. Xu and Yang (2013) studied a two-parallel-machines scheduling problem where one machine is periodically unavailable and jobs are non-preemptive. They applied the classical longest processing time first (LPT) rule and the list scheduling (LS) rule for minimizing the makespan. Computational experiments showed that the LPT beat the LS. Lei and Liu (2020) applied an artificial bee colony (ABC) algorithm with a division for solving distributed unrelated parallel machines scheduling problems

Table 1
Parameters of examples.

	Jobs										PM	
	0	1	2	3	4	5	6	7	8	9	T_i	I_i
M_0	4	8	3	2	3	8	4	9	3	2	7	10
M_1	6	7	9	9	3	9	2	8	3	2	14	9
M_2	3	4	7	5	7	5	9	6	9	5	10	12

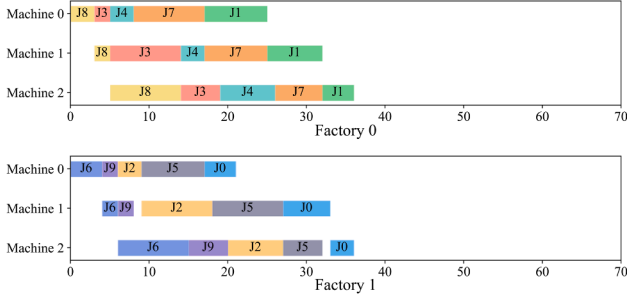


Fig 1. Gantt chart for an optimal DPFSP solution.

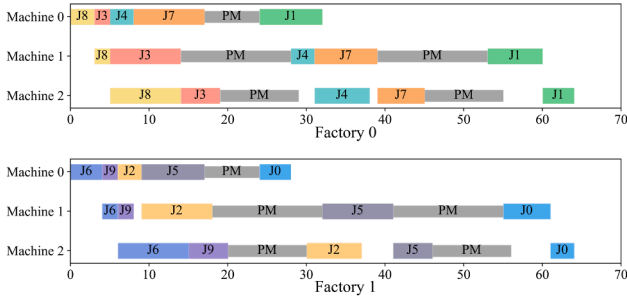


Fig 2. Gantt chart after applying the PM task scheduling criteria to the optimal DPFSP solution.

with PM operations. To generate solutions with high quality, they divided the whole swarm into one employed bee colony and three onlooker bee colonies, whose search strategy were different from each other.

As has been reviewed above, the DPFSP and the scheduling problems with PM operations have been widely studied in recent years. To the best of our knowledge, there is no existing literature on PM/DPFSP. Even though many high-performance algorithms have been developed, we are not aware of how well these existing algorithms perform on the PM/DPFSP. Therefore, in this study, we propose an effective MSIG algorithm and compare it with the state-of-the-art algorithms from the literature by a comprehensive comparison.

3. DPFSP with PM operation

The PM/DPFSP can be depicted as follows. The set of jobs N is to be assigned to the factory set G . In each factory, all the jobs are processed on the machine set M in the order from machine 1 to machine m following the same route. Each machine i , $i \in M$, can only process one job simultaneously and each job j , $j \in N$, can only be processed on one machine at one time. Each job j requires a certain amount of processing time on each machine i , which is described as p_{ij} . All the jobs and machines are available for the process at time 0. In order to ensure the maintainability and reliability of a production system, PM operations are processed on all the machines periodically. The maintenance operation performed on machine i is described as PM_i , $i \in M$. The processing interval of PM_i is denoted as I_i , which means that for each I_i units of

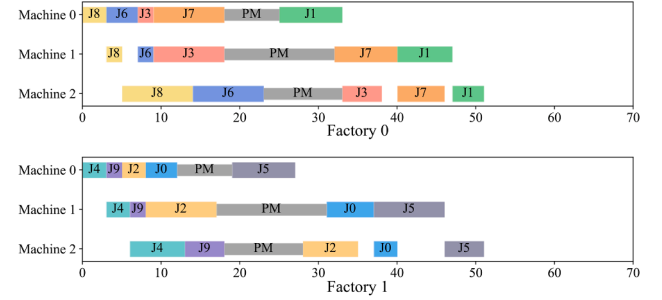


Fig 3. Gantt chart for an optimal solution considering the PM operations.

processing time, machine i must be stopped for PM. The processing time of PM_i is denoted as T_i . Once a job or a PM operation starts its processing on a machine, it cannot be interrupted.

The following example illustrates a solution of PM/DPFSP. Suppose that there is a set of ten jobs to be assigned to two factories each of which consists of two machines, that is, $m = 3$, $n = 10$ and $F = 2$. The processing time of the jobs and PM operations are shown in Table 1.

Fig. 1 shows an optimal solution for the example without considering the PM operator, i.e. the original DPFSP. We obtain the optimal solutions by enumerating all the feasible solutions. One of them is, the job sequences at factory 0 and factory 1 being $\{8, 3, 4, 7, 1\}$ and $\{6, 9, 3, 5, 0\}$ respectively. The makespan of this solution is 36. In Fig. 2, the scheduling criterion of the PM operations is applied to the previous optimal solution. It can be seen that the PM operations disrupted the tight schedule. There is plenty of idle time on machine 2 in each factory, which leads to larger completion time. Take job 2 in factory 1 on machine 1 for example to illustrate the PM operation, the accumulated total processing time before job 2 starts is $p_{1,6} + p_{1,9} = 2 + 2 = 4$, which is no larger than the interval time of PM operations on machine 2. While job 2 has a processing time of $p_{1,2} = 9$ on machine 1, resulting in an accumulated total processing time of 13 units, which is above $I_1 = 9$. Therefore, a PM operation is performed following job 2 on machine 1. The processing of job 5 is suspended until the PM operation finishes at time 32. Meanwhile, the accumulated total processing time on machine 1 is reset after maintenance. The same rule is applied to all the machines, thus producing makespan of 64.

Fig. 3 shows the Gantt chart for a solution to the previous problem but taking PM operations into consideration during scheduling. The solution consists of a job sequence $\{8, 6, 3, 7, 1\}$ in factory 0 and a job sequence $\{4, 9, 2, 0, 5\}$ in factory 1. As we can see, the new solution of the optimization has less idle time and fewer PM operations. In addition, the makespan is now 51, which implies a decrease in the initial solution in Fig. 2 of 20.3%. The explanation is given by the fact that the solution with PM operations considered is usually different from the one if the PM operations are not considered (Ruiz et al., 2007).

4. The proposed multi-start iterated greedy algorithm

In the literature, a high performing heuristic is often used to initialize the search for an IG algorithm. The main loop is to greedily repeat four operations (Ruiz & Stützle, 2007; Ruiz, Pan & Naderi, 2019):

Utilize the destruction to partially destroy the candidate solution. For our problem, some jobs will be removed from the solution, resulting in an incomplete solution and a group of leftover jobs.

Apply a greedy heuristic method to reinsert the removed jobs into the solution. A new complete solution will be generated in this construction phase.

Perform a local search to further optimize the new solution.

Use a simulated annealing-like acceptance criterion with a constant temperature to determine whether the new solution is accepted.

```

Procedure INEH2_dp
Calculate  $P_j = \sum_{i=1}^m p_{i,j}, \forall j \in N$ 
 $\lambda^{LPT} :=$  sort all jobs according to  $P_j$  in decreasing order
for  $f := 1$  to  $F$  do
 $\pi_f := \emptyset$ 
endfor
for  $step := 1$  to  $n$  do
 $j := \lambda^{LPT}[step]$ 
for  $f := 1$  to  $F$  do
 $P_{insert} :=$  all positions in  $\pi_f$  where job  $j$  can be inserted
 $P_{insert}^* :=$  randomly dropout positions from  $P_{insert}$  with probability  $k$ 
test to insert job  $j$  in  $P_{insert}^*$ 
 $C_{max}^f :=$  the lowest makespan obtained
 $p^f :=$  the position that generates the  $C_{max}^f$ 
endfor
 $f_{min} := \arg(\min_{f=1}^F (C_{max}^f))$ 
Insert job  $j$  in factory  $f_{min}$  at position  $p^{f_{min}}$ 
 $j^* := \arg(\min(P_{p^{f_{min}}+1}, P_{p^{f_{min}}-1}))$ 
Test to swap job  $j^*$  with all jobs in factory  $f_{min}$ 
Swap job  $j^*$  with the job resulting in the lowest makespan
endfor

```

Fig 4. Procedure of the INEH2_dp.

In this study, we record the number of consecutive iterations c in which the solution has not been improved. The solution will be abandoned once c is larger than a specific threshold α . The algorithm then restarts from the heuristic initialization phase. Such a multi-start strategy is adopted to prevent stagnation situations of the search due to insufficient diversification (Martí, Resende & Ribeiro, 2013). In the following, we describe the detailed phases of MSIG for PM/DPFSP.

4.1. Solution representation

To represent the job assignment to factories, a solution is represented by a two-dimensional array $\Pi = \{\pi_1, \pi_2, \dots, \pi_F\}$. There are F rows in the array, in which each row has a job sequence $\pi_f (f = 1, 2, \dots, F)$ consisting of a set of jobs assigned to the factory. Jobs in each sequence are sorted according to the processing order. For instance, the solution as shown in Fig. 3 can be represented as $\Pi = \{(8, 6, 3, 7, 1), (4, 9, 2, 0, 5)\}$. According to the PM strategy we take, once job sequences are determined, starting and ending times of PM operations on all machines are also fixed. So, there is no need to record PM operations in π_f . Such a sequence-based solution is widely used in the scheduling literature for its intuitive representation and convenience for programming (Arroyo, Leung & Tavares, 2019).

4.2. Heuristic method

As for the initialization procedure, we propose an improved NEH2 heuristic method with dropout operation (INEH2_dp for short) to produce a solution with a high level of quality. The NEH algorithm of Nawaz, Ensore and Ham (1983) has been regarded to be one of the most effective heuristic algorithms for the permutation flowshop scheduling problem (PFSP) with the makespan criterion (Ruiz & Maroto, 2005). Naderi and Ruiz (2010) extended NEH by considering the allocation of jobs to factories to fit for DPFSP and proposed the NEH1 and NEH2. The NEH2_en of Ruiz, Pan and Naderi (2019) is a further improvement on the NEH2 method which increases the computational complexity to obtain a better performance. In the NEH2, all the jobs are first arranged in decreasing order of the total processing time on m machines $P_j = \sum_{i=1}^m p_{i,j}, \forall j \in N$, resulting in the largest processing time (LPT) sequence. Then the jobs are orderly taken out from the sequence and placed at the best positions among all the F factories. The NEH2_en improved the NEH2 heuristic in which after the position of a new job is fixed, either the job on its predecessor or successor is extracted at random and reinserted in the best positions in the same factory. Inspired by the NEH2_en, we propose the INEH2_dp method to fit our problem. The

```

Procedure Destruction ( $\Pi = (\pi_1, \pi_2, \dots, \pi_F)$ )
 $\lambda_d := \emptyset$ 
for  $step := 1$  to  $d$  do
 $f_1, f_2 :=$  randomly select two factories
if ( $C_{max}(\pi_{f_1}) > C_{max}(\pi_{f_2})$ ) then
 $j :=$  randomly remove one job from  $\pi_{f_1}$ 
else
 $j :=$  randomly remove one job from  $\pi_{f_2}$ 
endif
Append job  $j$  to  $\lambda_d$ 
endfor
return  $\lambda_d$ 

```

Fig 5. Procedure of the destruction phase.

pseudo codes of INEH2_dp are given in Fig. 4.

In our INEH2_dp heuristic method, the new job taken out from the LPT sequence is not greedily tested in all the positions in all the factories. Instead, each possible position has a certain probability k of being ignored. We refer to this operator and its probability k as dropout and dropout rate, where k is a parameter to be calibrated. The purpose of the dropout operator is to enhance some stochastics to our heuristic so that a different initial solution with a high level of quality can be generated when the algorithm restarts. In addition, the predecessor and successor of the inserted job are compared by P_j . We select the job with a smaller P_j and test to swap it with all the other jobs in the same factory. Finally, the swapping operator producing the least makespan is chosen.

In the course of C_{max} calculation, each job on each machine in the sequence π_f is traversed one by one. There is an average of n/F jobs in each factory. This makes the complexity of C_{max} calculation in one factory equal to $O(nm/F)$. INEH2_dp evaluates a total of $[n(n + F)/2] \cdot F$ schedules, which obtains the complexity of $O(n^3m/F)$. Note that the dropout operator does not change the framework of the proposed heuristic, so it does not reduce the theoretical complexity.

4.3. Destruction phase

Destruction and construction procedures are the two core operators in the MSIG. In the destruction phase, we remove d jobs (d is an integer parameter to be adjusted) from all the factories and put them into a temporary sequence λ_d . In the PM/DPFSP, the larger makespan a factory generates, the greater impact it will have on the overall makespan. Therefore, it makes sense to bias the destruction towards factories with larger makespan. The tournament selection is applied in the destruction operator. We randomly pick out two factories and compare them in terms of makespan. Then a job is randomly removed from the factory with larger makespan. Note that the two random factories can be the

```

Procedure Construction ( $\Pi = (\pi_1, \pi_2, \dots, \pi_F), \lambda_d$ )
 $\lambda_d^{LPT} :=$  sort jobs in  $\lambda_d$  according to  $P_j$  in decreasing order
for  $step := 1$  to  $d$  do
 $j := \lambda_d^{LPT}[step]$ 
for  $f := 1$  to  $F$  do
Test job  $j$  in all possible positions in  $\pi_f$ 
 $C_{max}^f :=$  the lowest makespan obtained
 $p^f :=$  the position that generates the  $C_{max}^f$ 
endfor
 $f_{min} := \arg(\min_{f=1}^F (C_{max}^f))$ 
Insert job  $j$  in factory  $f_{min}$  at position  $p^{f_{min}}$ 
 $j^* := \arg(\min(P_{p^{f_{min}}+1}, P_{p^{f_{min}}-1}))$ 
for  $f := 1$  to  $F$  do
Test to swap job  $j^*$  with all jobs in  $\pi_f$ 
 $C_{max}^f :=$  the lowest makespan obtained
 $j^f :=$  the job that generates the  $C_{max}^f$ 
endfor
 $f_{min} := \arg(\min_{f=1}^F (C_{max}^f))$ 
Swap job  $j^*$  with  $j^f$  in  $\pi_{f_{min}}$ 
endfor

```

Fig 6. Procedure of the construction phase.


```

Procedure ISLS( $\Pi=(\pi_1, \pi_2, \dots, \pi_F)$ )
for  $f := 1$  to  $F$  do
  for  $i := 1$  to number of jobs in  $\pi_f$  do
    randomly select a job  $j$  from  $\pi_f$  without repetition
     $C_{max}^{ref} := C_{max}(\pi_f)$ 
    Test to swap job  $j$  with all other jobs in  $\pi_f$ 
     $C_{max}^*$  := the lowest makespan obtained
     $j^*$  := the job that generates the  $C_{max}^*$ 
    if ( $C_{max}^{ref} > C_{max}^*$ ) do
      Swap job  $j$  with  $j^*$ 
      break
    endif
  endfor
endfor

```

Fig 7. Procedure of the inter-factory swap local search.

```

Procedure ESLS( $\Pi=(\pi_1, \pi_2, \dots, \pi_F)$ )
 $C_{max}^* := \max_{f=1}^F (C_{max}(\pi_1), C_{max}(\pi_2), \dots, C_{max}(\pi_F))$ 
 $f_{key} :=$  the factory yielding  $C_{max}^*$ 
for  $i := 1$  to number of jobs in  $\pi_{f_{key}}$  do
  randomly select a job  $j$  from  $\pi_{f_{key}}$  without repetition
  for  $f := 1$  to  $F$  do
    Test to swap job  $j$  with all jobs in  $\pi_f$ 
     $C_{max}^f :=$  the lowest makespan obtained
     $j^f :=$  the job that generates the  $C_{max}^f$ 
  endfor
   $C_{max} := \min_{f=1}^F (C_{max}^f)$ 
  if  $C_{max} < C_{max}^*$  then
     $f_{min} := \arg(C_{max}^f)$ 
    Swap job  $j$  with  $j^{f_{min}}$  in  $\pi_{f_{min}}$ 
    break
  endif
endfor

```

Fig 8. Procedure of the external-key-factory swap local search.

```

Procedure EILS( $\Pi=(\pi_1, \pi_2, \dots, \pi_F)$ )
 $C_{max}^* := \max_{f=1}^F (C_{max}(\pi_1), C_{max}(\pi_2), \dots, C_{max}(\pi_F))$ 
 $f_{key} :=$  the factory yielding  $C_{max}^*$ 
for  $i := 1$  to number of jobs in  $\pi_{f_{key}}$  do
  randomly extract a job  $j$  from  $\pi_{f_{key}}$  without repetition
  for  $f := 1$  to  $F$  do
    Test to insert job  $j$  into all positions in  $\pi_f$ 
     $C_{max}^f :=$  the lowest makespan obtained
     $p^f :=$  the position that generates the  $C_{max}^f$ 
  endfor
   $C_{max} := \min_{f=1}^F (C_{max}^f)$ 
  if  $C_{max} < C_{max}^*$  then
     $f_{min} := \arg(C_{max}^f)$ 
    Insert job  $j$  into  $\pi_{f_{min}}$  at position  $p^{f_{min}}$ 
    break
  else then
    Return job  $j$  to the original position
  endif
endfor

```

Fig 9. Procedure of the external-key-factory swap local search.

same factory so that the factory with the least makespan also has a chance of being chosen. The procedure will be repeated for d times. The pseudo codes of destruction phases are given in Fig. 5.

4.4. Construction phase

As for the construction, we recombine the temporary sequence λ_d and the remaining partial solution π to reconstruct a new complete candidate solution. We found that the previously mentioned heuristic method can obtain good results with a little modification. Jobs in λ_d are rearranged by the total processing time P_j . We greedily insert the jobs into the best position among all the factories in the order of P_j , from the largest to the

```

Procedure LocalSearch( $\Pi=(\pi_1, \pi_2, \dots, \pi_F)$ )
while (improve) do
  improve := false
   $C_{max}^{ref} = C_{max}(\Pi)$ 
  ISLS( $\Pi$ )
  ESLS( $\Pi$ )
  EILS( $\Pi$ )
  if ( $C_{max}^{ref} \geq C_{max}(\Pi)$ ) do
    improve := true
  endif
end

```

Fig 10. Procedure of the complete local search.

smallest. Unlike in the INEH2_dp, where the selected predecessor or successor is tested to swap with jobs in the corresponding factory, the construction operator tests to swap the selected job with all the jobs in all the factories. It is assumed that each factory has an average of n/F jobs, so the complexity of testing to insert one job into all the possible positions and to swap one job with all the jobs are both $O(n^2m/F)$. The complexity of the construction phase is $O(n^2md/F)$. The pseudo codes of construction phases are given in Fig. 6.

4.5. Local search methods

After the destruction and construction phases, a local search is performed to further improve the solution. As to the permutation-based neighborhood structure, the swap and insertion operators are widely used in the local search phase (Fernandezviagas, Valente & Framinan, 2018). The swap operator is to interchange the positions of two jobs. The insertion operator removes a job from its original position and reinserts it into a different position. Based on the two operators, we propose three effective local search methods for the PM/DPFSP. The pseudo codes of the three local search methods are presented in Figs. 7–9, respectively.

Inter-factory Swap Local Search (ISLS): We test to swap each job in each factory in random order with the other jobs in the same factory. If an improvement swap is carried out, or all tests in a factory are completed, we end the search in the factory and begins the search in the next one. In the worst case where the solution is not improved, $(n/F)(n/F-1)/2$ swap operations will be tested in each factory. The ISLS terminates after all the factories get the swap search done. Therefore, the complexity of the ISLS in all F factories is $O(n^3m/F^2)$.

External-key-factory Swap Local Search (ESLS): In the PM/DPFSP, it is noted that makespan is determined by the factory with the largest makespan. Such a factory is referred to as the key factory, denoted as f_{key} . Jobs in f_{key} are picked up in a random order without repetition. Then we test to swap the selected job with all the other jobs in all the factories (including f_{key}) and then perform the best swap with the lowest C_{max} . The procedure terminates if an improvement is achieved, or all the jobs in f_{key} have been tested. It is assumed that there are averagely n/F jobs in f_{key} , so n^2/F swap operations will be tested at most. The complexity of ESLS is $O(n^3m/F^2)$.

External-key-factory Insertion Local Search (EILS): Similar to the ESLL, the EILS tests to reinsert jobs in f_{key} in all the factories (including f_{key}) and conduct the best insertion in order to minimize the C_{max} . Unlike ESLS, in which test of swapping jobs in two factories generates two new solutions and needs to calculate C_{max} result twice, the EILS only calculates one C_{max} result. Therefore, EILS shares the same complexity of $O(n^3m/F^2)$ with ESLS but theoretically has shorter computation time.

The local search phase is composed of the three proposed local search methods. It repeatedly calls the ISLS, the ESLS and the EILS until no better solution can be obtained. The pseudo codes of the complete local search procedures are presented in Fig. 10.

4.6. Acceptance criterion and restart strategy

The acceptance criterion is designed to avoid local best solutions. After the destruction and construction phases and the local search phase,

Procedure MSIG for PM/DPFSP

```

 $\Pi_{best} := \text{INEH2}()$ 
while (termination criterion not satisfied) do
   $c := 0$ 
   $\Pi := \text{INEH2}()$ 
  while ( $c < \alpha$ ) do
     $\Pi^* := \Pi$ 
     $\lambda_d := \text{Destruction}(\Pi^*)$ 
     $\text{Construction}(\Pi^*, \lambda_d)$ 
     $\text{LocalSearch}(\Pi^*)$ 
    if ( $C_{max}(\Pi^*) < C_{max}(\Pi)$ ) then
       $\Pi := \Pi^*$ 
    else if ( $\text{random} \leq \exp(-\frac{C_{max}(\Pi^*) - C_{max}(\Pi)}{\text{Temperature}})$ ) then
       $\Pi := \Pi^*$ 
    else then
       $\Pi^* := \Pi$ 
    endif
    if ( $C_{max}(\Pi^*) < C_{max}(\Pi_{best})$ ) then
       $\Pi_{best} := \Pi^*$ 
       $c := 0$ 
    else then
       $c := c + 1$ 
    endif
  endwhile
endwhile
return  $\Pi_{best}$ 

```

Fig 11. Procedure of the proposed MSIG algorithm.**Table 2**

Parameters of instances.

Number of jobs	n	{100, 200, 300, 400, 500}
Number of machines	m	{5, 8, 10}
Number of factories	F	{2, 4, 6}
Processing time of a job on each machine	P_{job}	[1, 100]
Processing time of PM operation on each machine	T_{PM}	[50, 150]
Interval time of PM operation on each machine	I_{PM}	[25 × n/F , 37.5 × n/F]

we apply a simulated annealing-like algorithm with constant temperature as an acceptance criterion. As is shown in the formula, a new solution Π^* with a worse result than the current solution Π has a certain probability p to be accepted, so it is hopeful to abandon a local best solution and keep searching for better results. The constant temperature is based on the particular instance and it is computed following the formula suggested by Osman and Potts (1989). Despite of its simplicity, such an acceptance criterion has been widely used in scheduling problems (Pan et al., 2019; Ruiz & Stützle, 2007; Ruiz, Pan & Naderi, 2019).

$$p = \exp\left(\frac{C_{max}(\Pi^*) - C_{max}(\Pi)}{\text{Temperature}}\right) \quad (1)$$

Table 3

RPD and total elapsed CPU time (in seconds) for the heuristics (minimum RPD values and minimum total CPU time are in bold).

		NEH2	NEH2_en	INEH2_dp (classified according to dropout rate)				
				0%	10%	20%	30%	40%
F	2	2.095	1.95	1.555	1.55	1.596	1.586	1.696
	4	4.566	4.344	3.797	3.759	3.85	3.862	3.998
	6	6.609	5.737	5.246	5.369	5.414	5.455	5.564
m	5	2.040	2.017	1.872	1.909	1.944	1.959	2.1
	8	4.875	4.441	3.859	3.922	4.001	3.961	4.131
	10	5.816	5.572	4.867	4.847	4.914	4.984	5.026
n	100	8.147	7.887	7.619	7.488	7.723	7.665	7.952
	200	5.29	4.914	4.214	4.36	4.478	4.454	4.57
	300	3.559	3.317	2.743	2.743	2.745	2.831	2.894
	400	2.381	2.235	1.742	1.836	1.796	1.824	1.903
	500	1.839	1.697	1.345	1.369	1.359	1.398	1.443
Average RPD		4.243	4.01	3.533	3.559	3.62	3.635	3.753
Total CPU time		72.9	99.69	100.7	91.87	83.02	75.22	68.1

$$\text{Temperature} = T \times \frac{\sum_{i=1}^m \sum_{j=1}^n P_{i,j}}{m \times n \times 10} \quad (2)$$

In addition, we think of the algorithm producing a local optimum if the global best solution has not been improved in α consecutive iterations. The solution is then abandoned and restarted from the heuristic initialization. During the whole progress, a number of candidate solutions are obtained, among which the best overall solution is the output of the algorithm. Such a restart strategy can effectively avoid the solution being trapped in a local optimum or cycling search (Lin et al., 2011; Zhang, Tu & Wu, 2019).

4.7. Framework of the proposed MSIG algorithm

The complete MSIG algorithm for minimizing the makespan of PM/DPFSP is summarized in Fig. 11.

5. Experimental results

In order to evaluate the algorithmic performance, we conducted numerical experiments. To the best of our knowledge, there is no existing instance of our problem. Therefore, we generated benchmarks based on the following parameters in Table 2.

The value of n , m , F and P_{job} are set based on the studies of Taillard (1993) and Pan et al. (2019) to cover the instances of all the scales. We also referred to Ruiz et al. (2007) to set up parameters for the PM operation. The minimum value of T_{PM} is set as 50 in case too short PM operation time has little impact on scheduling. The mean value of T_{PM} is two times of that of the average P_{job} . Meanwhile, in order to simplify the problem, the range of I_{PM} is set 50% to 75% of the average total processing time on each machine, so that only one PM operation is carried out on each machine during the whole manufacturing process.

In total, there are 45 different combinations of the number of factories F , the number of machines m and the number of jobs n . Each combination randomly generates five different instances, which results in $45 \times 5 = 225$ instances of varying scales. We have coded all the algorithms in C++ and compiled with Visual Studio 2019 with the x64 compiler with all optimization flags enable. All the experiments in this paper are performed on Windows Server 2012 on a PC with a Quad-core processor Intel E5-2640 2.4 GHz and 4 GB of RAM. The results of all the algorithms are measured by the relative percentage deviation (RPD) (Naderi & Ruiz, 2010; Ruiz & Stützle, 2007; Ruiz et al., 2007) and it is calculated as follows:

$$RPD = 100\% \times \frac{\text{Some}_{sol} - \text{Best}_{sol}}{\text{Best}_{sol}} \quad 3$$

where Some_{sol} is the C_{max} obtained by a certain algorithm on a given instance and Best_{sol} is the minimum C_{max} found in any experiment for the

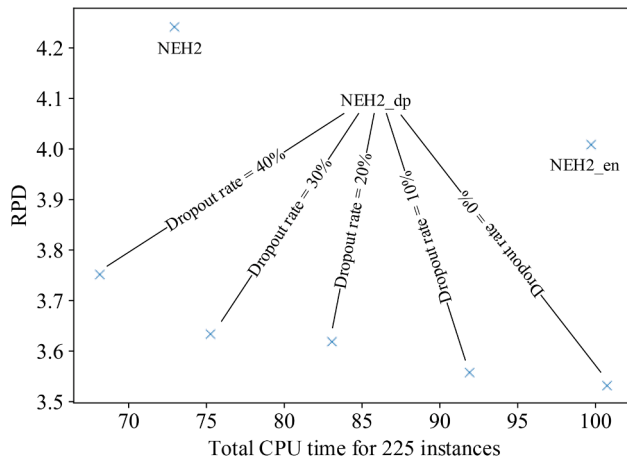


Fig 12. Mean RPD against total CPU time for the heuristics.

Table 4
Algorithms and their parameters.

Algorithms	Parameter setting
DDE	Population: 20; k: 0.1; Crossover rate: 0.1; Lambda: 0.01; Local search times: 40
HGA	Population: 10; Crossover rate: 0.1; Mutation rate: 0.01
IG2S	Destruction size: 4; Temperature: 0.3
ICG	Temperature: 2.5; I_{iter} : 2500; λ : 0.9; $I_{non-improving}$: 2.0; α_{min} : 2; η_{min} : 2
IDABC	Number of food sources: 10; limit: 10
IG	Destruction size: 3; Temperature: 0.2
MSIG	Dropout rate: 10%; Destruction size: 3; Temperature: 0.5; α : 1000

same instance. We carried out an analysis of variance (ANOVA) to determine whether the PRD results obtained by each algorithm are indeed significantly different. A 95% confidence level is employed in our experiments.

The stopping condition of an algorithm may be maximum computational time allowed, the maximum number of iterations, or the maximum number of two improvements. In this paper, a maximum elapsed CPU time, $t = C \times m \times n$ milliseconds, is set as the termination criterion, which has been commonly used in various scheduling problems (Pan et al., 2019). For the comparison experiments, we $C = 50$.

The proposed MSIG has three parameters: the number of destructed jobs d , constant temperature coefficient T for the simulated annealing-like acceptance criterion and the number of trials α after which the solution is restarted. We tested all reasonable combinations of k , d , T and α , where $k = \{0\%, 10\%, 20\%, 30\%, 40\%\}$, $d = \{2, 3, 4, 5\}$, $T = \{0.4, 0.5, 0.6, 0.7, 0.8\}$ and $\alpha = \{700, 800, 900, 1000, 1100\}$. Each combination of parameters was repeated five times on 45 instances covering all the combinations of the number of F , m and n . Following Ruiz and Stützle (2007), we analyzed the experimental data by ANOVA and finally set the parameters as follows: $k = 10\%$, $d = 3$, $T = 0.5$ and $\alpha = 1000$.

5.1. Comparison of constructive heuristics

In this section, we compare the presented INEH2_dp against the widely used NEH2 (Naderi & Ruiz, 2010) and NEH2_en (Ruiz, Pan & Naderi, 2019). In order to evaluate the impact of the dropout operation on the INEH2_dp heuristic method, we tested the dropout rate k at 5 values: 0%, 10%, 20%, 30% and 40%. The k being equal to 0% is a special case of the INEH2_dp, in which no dropout operation is carried out and the heuristic obtains a fixed solution on each instance. All the 225 instances are repeated 5 times on each algorithm and then averaged. The heuristics are evaluated in terms of both RPD and computational effort. We detail the average RPD and the total elapsed CPU times to solve all the 225 instances with 5 replications in Table 3. The average

Table 5

RPD of the algorithms for comparison grouped by instance size (the best values are in bold).

$F m n$	DDE	HGA	IG2S	ICG	IDABC	IG	MSIG
2 5 100	0.218	0.006	0.01	0.191	0	0	0
2 5 200	0.312	0.034	0.059	0.349	0.016	0.036	0.03
2 5 300	0.07	0.021	0.021	0.134	0.004	0.024	0.005
2 5 400	0.119	0.029	0.041	0.131	0.004	0.036	0.019
2 5 500	0.025	0	0.008	0.033	0.001	0.004	0
2 8 100	5.66	0.786	0.877	4.603	0.739	0.352	0.45
2 8 200	1.298	0.245	0.214	1.468	0.153	0.164	0.075
2 8 300	0.624	0.28	0.183	0.899	0.077	0.138	0.087
2 8 400	0.38	0.133	0.065	0.592	0.017	0.04	0.04
2 8 500	0.235	0.084	0.075	0.37	0.043	0.087	0.035
2 10 100	5.193	0.849	1.046	4.964	1.171	0.56	0.65
2 10 200	3.449	0.541	0.695	3.39	0.472	0.433	0.209
2 10 300	1.554	0.491	0.5	1.869	0.294	0.435	0.2
2 10 400	0.647	0.225	0.138	0.886	0.106	0.156	0.064
2 10 500	0.69	0.196	0.233	0.989	0.1	0.145	0.083
4 5 100	4.664	0.791	0.572	4.841	0.576	0.22	0.293
4 5 200	2.293	0.231	0.276	2.4	0.123	0.172	0.13
4 5 300	0.871	0.096	0.11	1.164	0.026	0.086	0.057
4 5 400	0.397	0.144	0.152	0.551	0.069	0.121	0.049
4 5 500	0.111	0.039	0.049	0.195	0.014	0.034	0.018
4 8 100	7.559	1.778	1.109	7.024	1.288	0.498	0.52
4 8 200	6.007	0.922	1.165	5.988	0.792	0.654	0.315
4 8 300	3.204	0.641	0.674	3.504	0.331	0.393	0.198
4 8 400	2.318	0.354	0.531	2.55	0.179	0.339	0.14
4 8 500	1.582	0.342	0.516	1.764	0.136	0.257	0.139
4 10 100	8.303	2.497	1.157	8.163	2.407	0.603	0.751
4 10 200	6.426	1.262	1.399	6.629	1.171	0.741	0.29
4 10 300	5.226	0.83	1.288	5.061	0.818	0.857	0.252
4 10 400	3.662	0.765	1.141	4.118	0.558	0.684	0.236
4 10 500	2.804	0.434	0.932	2.965	0.364	0.626	0.12
6 5 100	6.881	1.826	1.081	7.223	0.734	0.478	0.437
6 5 200	3.285	0.713	0.795	3.807	0.337	0.346	0.222
6 5 300	1.749	0.315	0.412	2.27	0.206	0.33	0.143
6 5 400	0.601	0.173	0.201	0.796	0.068	0.146	0.156
6 5 500	0.938	0.071	0.257	1.072	0.034	0.12	0.084
6 8 100	9.986	3.397	1.364	9.23	1.734	0.875	0.667
6 8 200	6.727	1.434	1.277	7.252	1.222	0.654	0.286
6 8 300	5.713	1.08	1.29	5.831	0.828	0.716	0.234
6 8 400	4.133	0.66	1.015	4.404	0.399	0.584	0.177
6 8 500	3.658	0.664	1.209	3.758	0.544	0.77	0.144
6 10 100	9.021	3.161	1.245	8.673	2.064	0.634	0.894
6 10 200	8.342	2.047	1.481	8.247	1.654	0.931	0.403
6 10 300	6.755	1.571	1.634	6.747	1.3	1.148	0.377
6 10 400	5.727	1.134	1.795	5.788	1.029	1.079	0.232
6 10 500	3.978	1.258	1.559	4.271	0.669	0.927	0.219
Average	3.409	0.768	0.708	3.492	0.553	0.414	0.225

Table 6

Average RPD of the algorithms for comparison grouped according to F , m , and n (best values are in bold).

		DDE	HGA	IG2S	ICG	IDABC	IG	MSIG
F	2	1.365	0.261	0.278	1.391	0.213	0.174	0.13
	4	3.695	0.742	0.738	3.794	0.59	0.419	0.234
	6	5.166	1.3	1.108	5.291	0.855	0.649	0.312
m	5	1.502	0.299	0.27	1.677	0.147	0.144	0.109
	8	3.939	0.853	0.771	3.949	0.566	0.435	0.234
	10	4.785	1.151	1.083	4.851	0.945	0.664	0.332
n	100	6.387	1.677	0.94	6.101	1.19	0.469	0.518
	200	4.238	0.826	0.818	4.392	0.66	0.459	0.218
	300	2.863	0.592	0.679	3.053	0.432	0.459	0.172
	400	1.998	0.402	0.564	2.202	0.27	0.354	0.124
	500	1.558	0.343	0.538	1.713	0.212	0.33	0.093
Global average		3.409	0.768	0.708	3.492	0.553	0.414	0.225

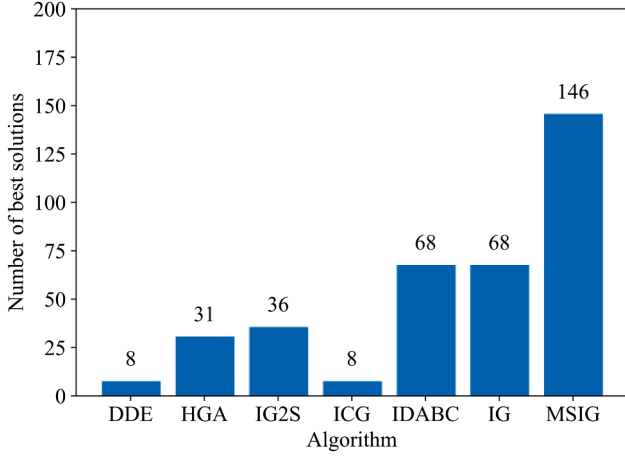
RPD values are classified according to F , m and n .

The heuristic methods are evaluated in terms of both average RPD results and computational effort. Fig. 12 shows a scatter plot of mean RPD against total CPU time. It can be seen that the simplest NEH2 is dominated by NEH2_en and INEH2_dp. The proposed INEH2_dp with dropout rate $k = 0\%$, 10% , 20% , 30% and 40% produce much smaller

Table 7

t-Test results for RPD results (the values less than 0.05 are in bold, indicating the significant difference between the algorithms).

t-test	DDE	HGA	IG2S	ICG	IDABC	IG	MSIG
DDE	–	–	–	–	–	–	–
HGA	1.56E–130	–	–	–	–	–	–
IG2S	3.97E–133	0.868084	–	–	–	–	–
ICG	0.38518	3.19E–154	1.10E–157	–	–	–	–
IDABC	4.90E–160	2.70E–12	3.00E–15	1.27E–187	–	–	–
IG	3.30E–172	1.07E–23	9.39E–31	8.97E–202	0.0054	–	–
MSIG	1.72E–200	2.06E–72	8.80E–96	3.31E–233	3.35E–35	3.34E–33	–

**Fig 13.** Number of best solutions of each instance generated by each algorithm.

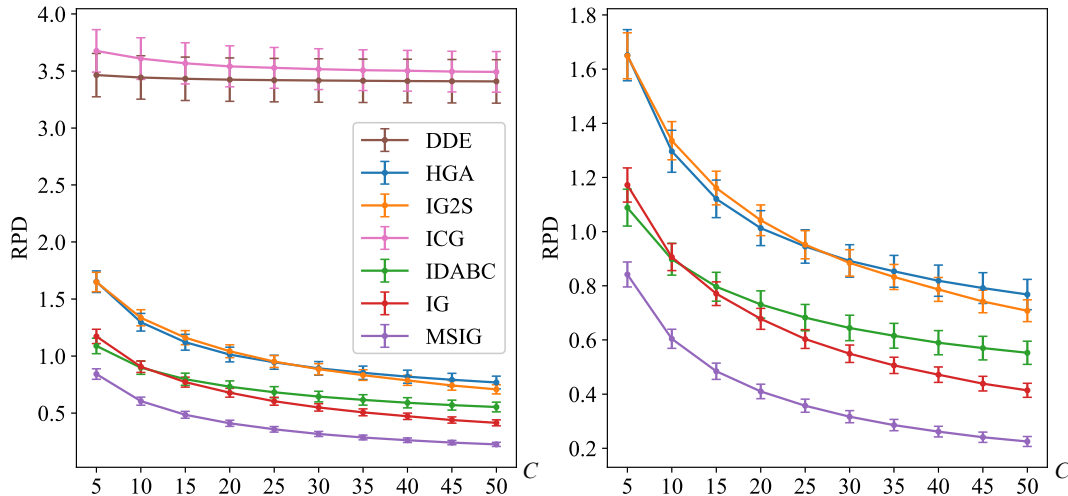
RPD values than the others. The smallest RPD value is obtained by the INEH2_dp ($k = 0\%$) with the largest total CPU time. The NEH2_en yields medium RPD values but shares similar CPU time with the INEH2_dp $k = 0\%$ due to the same algorithm structure. Meanwhile, INEH2_dp ($k = 10\%$) needs less total CPU time with a slight increase in RPD. The CPU time of INEH2_dp decreases as the dropout rate k increases. When k increases to 40%, INEH2_dp runs faster than the simplest NEH2 but performs significantly better. Recall that although the dropout operator does not reduce the theoretical complexity, it does decrease the computational time of the proposed INEH2_dp. As can be seen, INEH2_dp with $k = 0\%$, 10%, 20%, 30% and 40% are dominating NEH2 and NEH2_en. Considering RPD, total CPU time and solution diversity, we select INEH2_dp with $k = 10\%$ to initialize the solution.

5.2. Comparison of proposed algorithms

To validate the performance of the proposed MSIG, we compared it against six state-of-the-art algorithms from the literature. These are: discrete differential evolution (DDE) algorithm (Zhang, Xing & Cao, 2018), hybrid genetic algorithm (HGA) (Huang et al., 2019), two stage iterated greedy (IG2S) algorithm (Ruiz, Pan & Naderi, 2019), iterated cocktail greedy (ICG) algorithm (Lin & Ying, 2016), improved discrete artificial bee colony (IDABC) algorithm (Mao et al., 2020a) and iterated greedy (IG) (Mao et al., 2020b) algorithm, where DDE, HGA, IG2S and ICG are not designed for PM/DPFSP, so we reconstruct them for our problem. Parameters of all algorithms are calibrated as done in MSIG and summarized in Table 4. According to the above literature, the three population-based algorithms (DDE, HGA and IDABC) and the three iteration-based algorithms (IG2S, ICG and IG) respectively possess excellent global exploration and outstanding local exploitation. Therefore, they are selected to verify the proposed MSIG.

We perform five dependent runs for each algorithm to solve each of the 225 benchmarks with a given stop criterion $t = 50 \times m \times n$ milliseconds. The average RPD of each instance size $F|m|n$ is reported in Table 5. We also group the RPD values according to F , m and n in Table 6.

It can be clearly observed in Table 5 that the proposed MSIG performs best among all the algorithms in comparison. The overall RPD values of all the algorithms, from the smallest to the largest, are respectively 0.225% of MSIG, 0.414% of IG, 0.553% of IDABC, 0.708% of IG2S, 0.768% of HGA, 3.409% of DDE and 3.492% of ICG. DDE and ICG perform much worse than the other algorithms. The bad results obtained by DDE and ICG can be explained by the lack of an effective local search mechanism. In addition, the MSIG performs best on 29 out of 45 instance sizes, which is far more than the 12 of IDABC, the seven of IG and the one of HGA. Table 6 illustrates that the MSIG obtains the best result for almost all the groups with regarding to different numbers of factories F , different numbers of machines m and different numbers of jobs n , except for $n = 100$, in which average RPD of IG is slightly smaller

**Fig 14.** Means plots for algorithm comparison. On the right plot, the DDE and the ICG are removed.

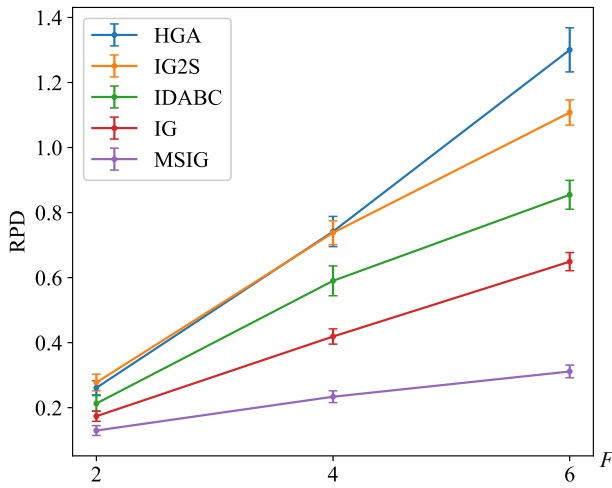


Fig 15. Means plots for the interaction between algorithm type and F .

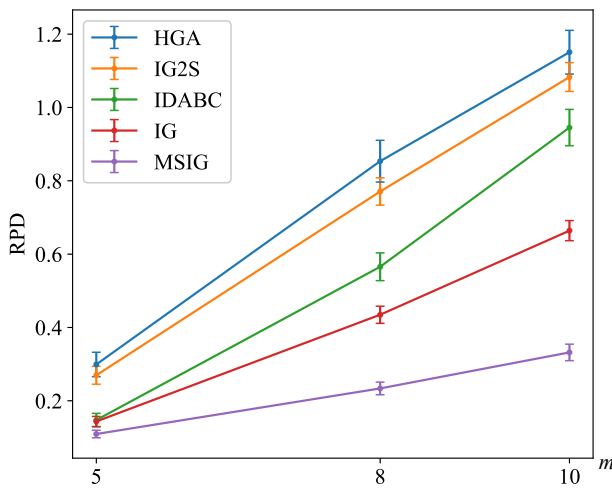


Fig 16. Means plots for the interaction between algorithm type and m .

than that of MSIG.

In addition, Table 7 concludes a t -test with a significance level equal to 5% for all the results yielded by each pair of the algorithms. According to Table 7, it is clear that there is no significant difference between the pair of HGA and IG2S and the pair of DDE and ICG. All the other algorithm pairs generate significantly different results. Therefore, we can divide the algorithms into five homogenous groups, with no significant differences within each group. In order of average RPD values, from the smallest to the largest, the groups are sorted as follows: {MSIG}, {IG}, {IDABC}, {IG2S, HGA}, and {DDE, ICG}.

In Fig. 13, we count the number of best solutions of each instance obtained by each algorithm. It can be observed that the MSIG generates the most number of best solutions among all the competitive algorithms. There are 146 best solutions generated by MSIG out of a total of 225 instances. Though IG is significantly better than IDABC in terms of average RPD results, they are tied for second place with 68 best solutions. IG2S, HGA, DDE and ICG generate 36, 31, 8 and 8 best solutions respectively.

Fig. 14 shows the average RPD values generated by all the algorithms at the elapsed CPU time $t = 50 \times m \times n$ milliseconds. We record the makespan of the best solution over a period of $5 \times m \times n$ milliseconds to provide an overall picture of the algorithms. Meanwhile, following Zou et al. (Zou et al., 2020) and many others, a multifactor ANOVA is carried out to confirm whether the RPD values of each algorithm at each moment are indeed significantly different. As can be seen, the mean

plots show that the proposed MSIG performs significantly better than the other algorithms all the time throughout the progress. DDE and ICG have the worst optimizations of all the algorithms because their RPD results are the largest and decrease very little as computational time increases. Furthermore, DDE and ICG have more parameters than the others, which increases calibration complexity. IG performs a bit worse than IDABC at the beginning, but it statistically exceeds IDABC after $35 \times m \times n$ milliseconds and further expands the gap between them as time goes on. Throughout the whole process, there is no significant difference between the RPD results of HGA and IG2S.

Next, we plot the interactions of algorithms type and the parameters of instances F , m and n in Figs. 15 and 16 based on the RPD results at $t = 50 \times m \times n$ milliseconds. The results are again analyzed by ANOVA where F , m , and n are considered as non-controllable factors. DDE and ICG are removed from these plots as they are clearly worse than the other algorithms. We can see that when the number of factories F and the number of machines m change, the average RPD values of each algorithm have the same changing trend, while the MSIG has the minimum variation. In other words, it is proved that MSIG has better robustness for the instances of a different scale than the other competitive algorithms. Fig. 16 shows that the population-based algorithms, HGA and IDABC, perform relatively poorly in the small-scale n of the instances while get better rapidly as n increases. When n increases to 500, HGA gains no significantly worse RPD results than the overall second-best IG. And IDABC even far exceeds IG. By contrast, the greedy iteration-based algorithms, MSIG, IG and IG2S, generate more stable RPD results when n changes. In general, the proposed MSIG is superior to the other compared algorithms in terms of both RPD results and stability when the parameters of instances change.

Finally, it can be safely summarized that the MSIG is the most effective algorithm for solving the PM/DPFSP with makespan criterion among all the compared algorithms. The MSIG outperforms the other algorithms in terms of both RPD results and robustness.

6. Conclusions and future research

Distributed scheduling in multi-factory production has been a hot research topic in the past decades. However, scheduling problems with PM operations are seldom investigated. In this study, the distributed permutation flowshop scheduling problem with preventive maintenance (PM/DPFSP) is considered. We proposed a simple but effective multi-start iterated greedy (MSIG) to minimize the makespan. The MSIG uses an improved NEH2 heuristic algorithm with a dropout operation to initialize the solution. After that, destruction and construction phases are greedily iterated. During the destruction phase, a couple of jobs are removed from the complete candidate solution. After that, the construction phase applies a constructive heuristic to reconstruct the solution. In attempting to obtain optimal solutions, a local search basing on insertion and swap operations is also presented. Once a feasible solution is completed, an acceptance criterion determines whether the incumbent solution will be replaced by the new one. In addition, a solution will be abandon after a certain number of iterations without improvement. Then the algorithm restarts with the heuristic method. We carried out experiments based on randomly generated instances to evaluate the effectiveness of the proposed MSIG. The experimental results have indicated that the MSIG performs much better than the existing algorithms in the literature.

In the near future, we will still focus on PM/DPFSP. Despite the avoidance of the remaining local optima, the multi-start strategy does not take full advantage of existing solutions. We are going to further study the restarting method to promote efficiency. In addition, we are also interested in other objectives in PM/DPFSP (e.g. total flow time, total tardiness) which are common in practice.

CRediT authorship contribution statement

Jia-yang Mao: Software, Data curation, Writing - original draft.
Quan-ke Pan: Conceptualization, Resources, Writing - review & editing, Supervision.
Zhong-hua Miao: Resources.
Liang Gao: Resources.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This research is partially supported by the National Natural Science Foundation of China 61973203 and the National Natural Science Fund for Distinguished Young Scholars of China 51825502 and Shanghai Key Laboratory of Power station Automation Technology. We are grateful to the Computing Center of Shanghai University and Prof. Zipeng Shan for providing the cloud computing cluster.

References

- Arroyo, J. E. C., Leung, J.-T., & Tavares, R. G. (2019). An iterated greedy algorithm for total flow time minimization in unrelated parallel batch machines with unequal job release times. *Engineering Applications of Artificial Intelligence*, 77, 239–254. <https://doi.org/10.1016/j.engappai.2018.10.012>
- Bargaoui, H., Belkahl, Driss, O., & Ghédira, K. (2017). A novel chemical reaction optimization for the distributed permutation flowshop scheduling problem with makespan criterion. *Computers & Industrial Engineering*, 111, 239–250. <https://doi.org/10.1016/j.cie.2017.07.020>
- Behnamian, J., & Ghomi, S. M. T. F. (2016). A survey of multi-factory scheduling. *Journal of Intelligent Manufacturing*, 27(1), 231–249. <https://doi.org/10.1007/s10845-014-0890-y>
- Birolini, A. (2013). *Reliability engineering: Theory and practice*. [S.l.]: Springer Science & Business Media.
- Deng, J., & Wang, L. (2017). A competitive memetic algorithm for multi-objective distributed permutation flow shop scheduling problem. *Swarm and Evolutionary Computation*, 32, 121–131. <https://doi.org/10.1016/j.swevo.2016.06.002>
- Fernandez-Viagas, V., Perez-Gonzalez, P., & Framinan, J. M. (2018). The distributed permutation flow shop to minimise the total flowtime. *Computers & Industrial Engineering*, 118, 464–477. <https://doi.org/10.1016/j.cie.2018.03.014>
- Fernandez-Viagas, V., & Framinan, J. M. (2015). A bounded-search iterated greedy algorithm for the distributed permutation flowshop scheduling problem. *International Journal of Production Research*, 53(4), 1111–1123. <https://doi.org/10.1080/00207543.2014.948578>
- Fernandez-Viagas, V., Valente, J. M. S., & Framinan, J. M. (2018). Iterated-greedy-based algorithms with beam search initialization for the permutation flowshop to minimise total tardiness. *Expert Systems with Applications*, 94, 58–69. <https://doi.org/10.1016/j.eswa.2017.10.050>
- Gao, J., & Chen, R. (2011). A hybrid genetic algorithm for the distributed permutation flowshop scheduling problem. *International Journal of Computational Intelligence Systems*, 4(4), 497–508. <https://doi.org/10.1080/18756891.2011.9727808>
- Gao, J., Chen, R., & Deng, W. u. (2013). An efficient tabu search algorithm for the distributed permutation flowshop scheduling problem. *International Journal of Production Research*, 51(3), 641–651. <https://doi.org/10.1080/00207543.2011.644819>
- Huang, J., Pan, Q., & Chen, Q. (2019). A hybrid genetic algorithm for the distributed permutation flowshop scheduling problem with sequence-dependent setup times. *IOP Conference Series: Materials Science and Engineering*, 646(1).
- Huang, J.-P., Pan, Q.-K., & Gao, L. (2020). An effective iterated greedy method for the distributed permutation flowshop scheduling problem with sequence-dependent setup times. *Swarm and Evolutionary Computation*, 59, 100742. <https://doi.org/10.1016/j.swevo.2020.100742>
- Huang, J.-P., Pan, Q.-K., Miao, Z.-H., & Gao, L. (2021). Effective constructive heuristics and discrete bee colony optimization for distributed flowshop with setup times. *Engineering Applications of Artificial Intelligence*, 97, 104016. <https://doi.org/10.1016/j.engappai.2020.104016>
- Jing, X.-L., Pan, Q.-K., Gao, L., & Wang, Y.-L. (2020). An effective Iterated Greedy algorithm for the distributed permutation flowshop scheduling with due windows. *Applied Soft Computing*, 96, 106629. <https://doi.org/10.1016/j.asoc.2020.106629>
- Lei, D., & Liu, M. (2020). An artificial bee colony with division for distributed unrelated parallel machine scheduling with preventive maintenance. *Computers & Industrial Engineering*, 141, 106320. <https://doi.org/10.1016/j.cie.2020.106320>
- Lin, S.-W., Ying, K.-C., & Huang, C.-Y. (2013). Minimising makespan in distributed permutation flowshops using a modified iterated greedy algorithm. *International Journal of Production Research*, 51(16), 5029–5038. <https://doi.org/10.1080/00207543.2013.790571>
- Lin, S.-W., & Ying, K.-C. (2016). Minimizing makespan for solving the distributed no-wait flowshop scheduling problem. *Computers & Industrial Engineering*, 99, 202–209. <https://doi.org/10.1016/j.cie.2016.07.027>
- Lin, S.-W., Ying, K.-C., Lu, C.-C., & Gupta, J. N. D. (2011). Applying multi-start simulated annealing to schedule a flowline manufacturing cell with sequence dependent family setup times. *International Journal of Production Economics*, 130(2), 246–254. <https://doi.org/10.1016/j.ijpe.2011.01.004>
- Maccarthy, B. L., & Liu, J. (1993). Addressing the gap in scheduling research: A review of optimization and heuristic methods in production scheduling. *International Journal of Production Research*, 31(1), 59–79. <https://doi.org/10.1080/00207549308956713>
- Mao, J., Hu, X. L., Pan, Q. K., Miao, Z., He, C. & Tasgetiren, M. F. (2020a). An improved discrete artificial bee colony algorithm for the distributed permutation flowshop scheduling problem with preventive maintenance. Chinese Control Conference, CCC, 2020-July, 1679–1684.
- Mao, J., Hu, X. L., Pan, Q. K., Miao, Z., He, C. & Tasgetiren, M. F. (2020b). An iterated greedy algorithm for the distributed permutation flowshop scheduling problem with preventive maintenance to minimize total flowtime. Chinese Control Conference, CCC, 2020-July, 1507–1512.
- Martí, R., Resende, M. G. C., & Ribeiro, C. C. (2013). Multi-start methods for combinatorial optimization. *European Journal of Operational Research*, 226(1), 1–8. <https://doi.org/10.1016/j.ejor.2012.10.012>
- Meng, T., Pan, Q.-K., & Wang, L. (2019). A distributed permutation flowshop scheduling problem with the customer order constraint. *Knowledge-Based Systems*, 184, 104894. <https://doi.org/10.1016/j.knsys.2019.104894>
- Naderi, B., & Ruiz, R. (2014). A scatter search algorithm for the distributed permutation flowshop scheduling problem. *European Journal of Operational Research*, 239(2), 323–334. <https://doi.org/10.1016/j.ejor.2014.05.024>
- Naderi, B., & Ruiz, R. (2010). The distributed permutation flowshop scheduling problem. *Computers & Operations Research*, 37(4), 754–768. <https://doi.org/10.1016/j.cor.2009.06.019>
- Naderi, B., Zandieh, M., & Aminnayeri, M. (2011). Incorporating periodic preventive maintenance into flexible flowshop scheduling problems. *Applied Soft Computing*, 11(2), 2094–2101. <https://doi.org/10.1016/j.asoc.2010.07.008>
- Nawaz, M., Ensore, E. E., Jr, & Ham, I. (1983). A heuristic algorithm for the m-machine, n-job flow-shop sequencing problem. *Omega*, 11(1), 91–95. [https://doi.org/10.1016/0305-0483\(83\)90088-9](https://doi.org/10.1016/0305-0483(83)90088-9)
- Osman, I. H., & Potts, C. (1989). Simulated Annealing for Permutation Flow-shop Scheduling. *Omega*, 17(6), 551–557.
- Pan, Q., Gao, L., Wang, L., Liang, J. J., & Li, X. (2019). Effective Heuristics and Metaheuristics to Minimize Total Flowtime for the Distributed Permutation Flowshop Problem. *Expert Systems with Applications*, 124, 309–324.
- Pan, Q.-K., Gao, L., Xin-Yu, L. i., & Jose, F. M. (2019). Effective constructive heuristics and meta-heuristics for the distributed assembly permutation flowshop scheduling problem. *Applied Soft Computing*, 81, 105492. <https://doi.org/10.1016/j.asoc.2019.105492>
- Rausand M., & Høyland A. (2003). *System Reliability Theory: Models, Statistical Methods, and Applications*. [S.l.]: John Wiley & Sons.
- Rifai, A. P., Nguyen, H.-T., & Dawal, S. Z. M. (2016). Multi-objective adaptive large neighborhood search for distributed reentrant permutation flow shop scheduling. *Applied Soft Computing*, 40, 42–57. <https://doi.org/10.1016/j.asoc.2015.11.034>
- Ruiz, R., Carlos García-Díaz, J., & Maroto, C. (2007). Considering scheduling and preventive maintenance in the flowshop sequencing problem. *Computers & Operations Research*, 34(11), 3314–3330. <https://doi.org/10.1016/j.cor.2005.12.007>
- Ruiz, R., & Maroto, C. (2005). A comprehensive review and evaluation of permutation flowshop heuristics. *European Journal of Operational Research*, 165(2), 479–494. <https://doi.org/10.1016/j.ejor.2004.04.017>
- Ruiz, R., Pan, Q.-K., & Naderi, B. (2019). Iterated Greedy methods for the distributed permutation flowshop scheduling problem. *Omega*, 83, 213–222. <https://doi.org/10.1016/j.omega.2018.03.004>
- Ruiz, R., & Stützle, T. (2007). A simple and effective iterated greedy algorithm for the permutation flowshop scheduling problem. *European Journal of Operational Research*, 177(3), 2033–2049. <https://doi.org/10.1016/j.ejor.2005.12.009>
- Taillard, E. (1993). Benchmarks for basic scheduling problems. *European Journal of Operational Research*, 64(2), 278–285. [https://doi.org/10.1016/0377-2217\(93\)90182-M](https://doi.org/10.1016/0377-2217(93)90182-M)
- Wang, K., Huang, Y., & Qin, H. u. (2016). A fuzzy logic-based hybrid estimation of distribution algorithm for distributed permutation flowshop scheduling problems under machine breakdown. *Journal of the Operational Research Society*, 67(1), 68–82. <https://doi.org/10.1057/jors.2015.50>
- Wang, J.-J., & Wang, L. (2020). A knowledge-based cooperative algorithm for energy-efficient scheduling of distributed flow-shop. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 50(5), 1805–1819. <https://doi.org/10.1109/TSMC.2017.2788879>
- Wang, S.-Y., & Wang, L. (2016). An estimation of distribution algorithm-based memetic algorithm for the distributed assembly permutation flow-shop scheduling problem. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 46(1), 139–149. <https://doi.org/10.1109/TSMC.2015.2416127>
- Wang, S.-Y., Wang, L., Liu, M., & Xu, Y. e. (2013). An effective estimation of distribution algorithm for solving the distributed permutation flow-shop scheduling problem. *International Journal of Production Economics*, 145(1), 387–396. <https://doi.org/10.1016/j.ijpe.2013.05.004>
- Wang, S., & Yu, J. (2010). An effective heuristic for flexible job-shop scheduling problem with maintenance activities. *Computers & Industrial Engineering*, 59(3), 436–447. <https://doi.org/10.1016/j.cie.2010.05.016>
- Wong, C. S., Chan, F. T. S., & Chung, S. H. (2013). A joint production scheduling approach considering multiple resources and preventive maintenance tasks.

- International Journal of Production Research*, 51(3), 883–896. <https://doi.org/10.1080/00207543.2012.677070>
- Xu, D., & Yang, D.-L. (2013). Makespan minimization for two parallel machines scheduling with a periodic availability constraint: Mathematical programming model, average-case analysis, and anomalies. *Applied Mathematical Modelling*, 37 (14–15), 7561–7567. <https://doi.org/10.1016/j.apm.2013.03.001>
- Xu, Y.e., Wang, L., Wang, S., & Liu, M. (2014). An effective hybrid immune algorithm for solving the distributed permutation flow-shop scheduling problem. *Engineering Optimization*, 46(9), 1269–1283. <https://doi.org/10.1080/0305215X.2013.827673>
- Yang, D.-L., Hsu, C.-J., & Kuo, W.-H. (2008). A two-machine flowshop scheduling problem with a separated maintenance constraint. *Computers & Operations Research*, 35(3), 876–883. <https://doi.org/10.1016/j.cor.2006.04.007>
- Zhang, G., Xing, K., & Cao, F. (2018). Discrete differential evolution algorithm for distributed blocking flowshop scheduling with makespan criterion. *Engineering Applications of Artificial Intelligence*, 76, 96–107. <https://doi.org/10.1016/j.engappai.2018.09.005>
- Zhang, W., Tu, J., & Wu, L. (2019). A multi-start iterated greedy algorithm for the minimum weight vertex cover P3 problem. *Applied Mathematics and Computation*, 349, 359–366. <https://doi.org/10.1016/j.amc.2018.12.067>
- Zhao, F., Zhao, L., Wang, L., & Song, H. (2020). An ensemble discrete differential evolution for the distributed blocking flowshop scheduling with minimizing makespan criterion. *Expert Systems with Applications*, 160, 113678. <https://doi.org/10.1016/j.eswa.2020.113678>
- Zou, W.-Q., Pan, Q.-K., Meng, T., Gao, L., & Wang, Y.-L. (2020). An effective discrete artificial bee colony algorithm for multi-AGVs dispatching problem in a matrix manufacturing workshop. *Expert Systems with Applications*, 161, 113675. <https://doi.org/10.1016/j.eswa.2020.113675>
- Lu, C., Gao, L., Gong, W., Hu, C., Yan, X., & Li, X. (2021). Sustainable scheduling of distributed permutation flow-shop with non-identical factory using a knowledge-based multi-objective memetic optimization algorithm. *Swarm and Evolutionary Computation*, 60, 100803. <https://doi.org/10.1016/j.swevo.2020.100803>
- Meng, T., & Pan, Q. K. (2021). A distributed heterogeneous permutation flowshop scheduling problem with lot-streaming and carryover sequence-dependent setup time. *Swarm and Evolutionary Computation*, 60, 100804. <https://doi.org/10.1016/j.swevo.2020.100804>